# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING) Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

| | | | |
|---|---|---|---|
| **Course Code:** | CSA06 | **Course Title:** | Design and Analysis of Algorithms |
| **CO Assessed:** | CO2 – Manipulation (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 2 – Scenario Based Assessment |
| **Weightage:** | 20% of CIA | **Total Marks:** | 100 |
| **CO2 Statement:** | *Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)* | | |
| **Student Name:** | **Sameer Ahmed G** | **Reg. No.:** | **192421154** |
| **Section / Batch:** | **B Tech IT** | **Date:** | **28-07-2026** |

**Instructions to Students**

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

| Question Type | Focus Area | Questions |
|---|---|---|
| **Scenario Based Assessment** | Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems | Q1 |

## SECTION A – SCENARIO BASED ASSESSMENT

**Code of Conduct**

*I Sameer Ahmed G (192421154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Sameer Ahmed G

## RUBRICS FOR CALCULATION

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Understanding & Context Analysis | 25% | Thoroughly understands the scenario with accurate interpretation of all requirements | Clearly understands the scenario with minor gaps | Basic understanding; some aspects of the scenario unclear | Poor understanding or misinterpretation of the scenario |
| Application of Concepts | 30% | Applies appropriate concepts effectively to solve complex real world problems | Applies relevant concepts correctly with minor errors | Applies basic concepts but with limited accuracy | Fails to apply appropriate concepts |
| Analytical & Decision Making Skills | 20% | Demonstrates strong analysis and selects optimal solutions with justification | Good analysis with reasonable solutions | Limited analysis; solutions lack justification | Poor or no analysis; incorrect decisions |
| Solution Design & Approach | 15% | Well-structured, innovative, and feasible solution approach | Logical and structured solution with minor gaps | Basic solution with limited structure or feasibility | Unclear or incorrect solution approach |
| Clarity, Justification & Presentation | 10% | Clear, well-justified explanation with strong reasoning | Clear explanation with some justification | Limited clarity and weak justification | Poorly explained with no justification |

## Marks Evaluation:

| Criteria | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Understanding & Context Analysis | 25% | | | | |
| Application of Concepts | 30% | | | | |
| Analytical & Decision-Making Skills | 20% | | | | |
| Solution Design & Approach | 15% | | | | |
| Clarity, Justification & Presentation | 10% | | | | |

| **Total Marks (100):** | |
|---|---|

| Faculty In-charge | Course Coordinator | Head of Department |
|---|---|---|
| Name: | Name: | Name: |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# Binary Search on Employee Payroll System

**Aim**

To implement Binary Search for searching employee IDs in a payroll system, analyze the effect of unsorted data on search performance, and explain the importance of maintaining sorted datasets.

**Problem Statement**

A payroll management system stores employee IDs in sorted order to enable quick searching using the Binary Search algorithm. However, frequent insertion of new employee records may disturb the sorted order. Analyze how Binary Search works, study the impact of unsorted data on searching efficiency, and explain why maintaining sorted datasets is essential.

**Algorithm**

1. Store employee IDs in an array.
2. Keep the array sorted.
3. Accept an employee ID to search.
4. Set
    - low = 0
    - high = n − 1
5. Repeat until low ≤ high
    - Find middle element.
    - If key equals middle element, return its position.
    - If key is smaller, search left half.
    - Else search right half.
6. If not found, display "Employee ID not found."

**Python Program**

```
# Binary Search - Payroll System

def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1


# Sorted Employee IDs
employee_ids = [1001, 1005, 1010, 1015, 1020, 1025, 1030, 1035, 1040, 1045]

print("Payroll System")
print("--------------------------")
print("Employee IDs:", employee_ids)

key = int(input("Enter Employee ID to search: "))

result = binary_search(employee_ids, key)

if result != -1:
    print("\nEmployee ID Found!")
    print("Employee ID:", key)
    print("Position:", result + 1)
else:
    print("\nEmployee ID Not Found!")
```

**Sample Input**
1025

**Sample Output**
Employee IDs:
1001 1005 1010 1015 1020 1025 1030 1035 1040 1045

Enter Employee ID to search:
1025

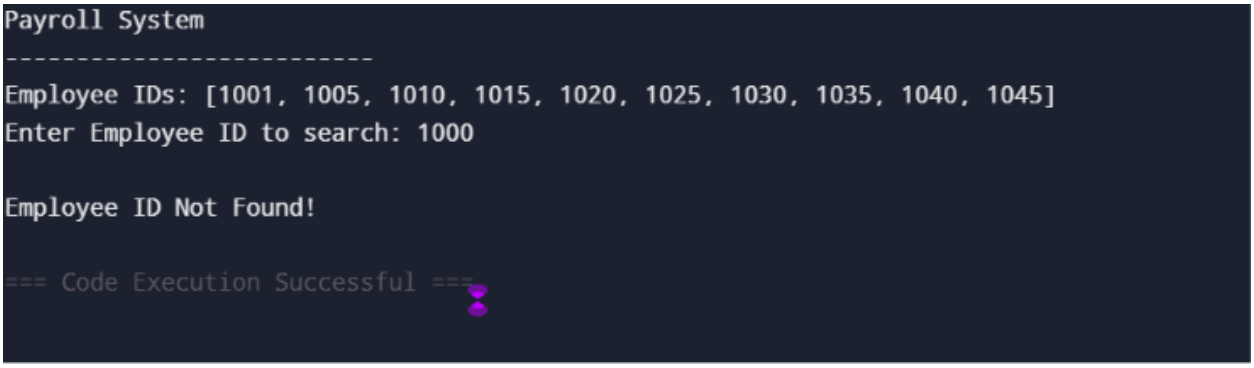
Employee ID found at position : 6

```
Payroll System
--------------------------
Employee IDs: [1001, 1005, 1010, 1015, 1020, 1025, 1030, 1035, 1040, 1045]
Enter Employee ID to search: 1005

Employee ID Found!
Employee ID: 1005
Position: 2

=== Code Execution Successful ===
```

**Sample Output 2 (Not Found)**
Input
1090
Output
Employee IDs:
1001 1005 1010 1015 1020 1025 1030 1035 1040 1045

Enter Employee ID to search:
1090

Employee ID not found.




**a) Explain how Binary Search works**

Binary Search is an efficient searching algorithm that works only on sorted datasets. It repeatedly divides the search space into two halves by comparing the target value with the middle element of the array. If the middle element matches the target, the search stops. If the target is smaller, the algorithm continues searching in the left half; otherwise, it searches the right half. This process continues until the element is found or the search interval becomes empty. The time complexity of Binary Search is **O(log n)**, making it much faster than Linear Search for large datasets.

**b) Analyze the impact of unsorted data on performance**

Binary Search depends entirely on the array being sorted. When employee IDs become unsorted because of frequent additions, the algorithm cannot correctly determine whether to search the left or right half. As a result, Binary Search may return incorrect results or fail to locate an existing employee ID. To perform searching correctly on unsorted data, the system must either sort the data first using a sorting algorithm with a complexity of approximately **O(n log n)** or use Linear Search, which has a time complexity of **O(n)**. Therefore, unsorted data significantly reduces search efficiency.

**c) Importance of maintaining sorted datasets**

Maintaining sorted employee IDs ensures that Binary Search can be applied effectively. A sorted dataset provides faster searching, reduces processing time, improves overall payroll performance, and enables quick retrieval of employee information even when the database becomes very large. Although maintaining sorted order requires additional effort during insertion, it greatly improves search efficiency and system responsiveness. Hence, sorted datasets are essential in payroll systems where frequent searches are performed.

**Complexity Analysis**

| Operation | Time Complexity |
|---|---|
| Binary Search | O(log n) |
| Best Case | O(1) |
| Worst Case | O(log n) |
| Space Complexity | O(1) |

**Decision Making**

Binary Search is selected because payroll systems perform employee searches frequently. Although inserting new employee records may require maintaining sorted order, the improvement in search speed outweighs the insertion cost. Therefore, Binary Search is the most suitable technique for payroll systems with frequent lookup operations.

**Conclusion**

The Binary Search algorithm efficiently locates employee IDs in a sorted payroll database with logarithmic time complexity. However, if new employee records disturb the sorted order, Binary Search becomes ineffective. Maintaining sorted datasets ensures faster searching, improved performance, and reliable payroll management.